Faculty of Media Engineering and Technology
German University in Cairo
Dr. Nourhan Ehab

CSEN 903: Advanced Computer Lab, Winter 2026
Lab 2 Manual: Machine Learning Foundations

Part 1: Read and Watch Before the Lab

In the previous lab, we focused on the **data cleaning and preprocessing** phase. This step was essential to prepare the Titanic dataset so that it could be effectively used in machine learning (ML) models, an important stage since well-prepared data often has a greater impact on model performance than the choice of algorithm itself.

Machine learning is at the heart of modern AI systems, and it comes in different flavors, including some major ones like:

- **Supervised learning:** models learn from labeled data (input + expected output).
- **Unsupervised learning:** models find hidden patterns or groupings in unlabeled data.
- **Semi-supervised learning:** models learn from a small amount of labeled data combined with a large amount of unlabeled data, which is especially useful when labeling is expensive or time-consuming.

In this lab, we will focus on **supervised learning,** which can be used for 2 types of problems:

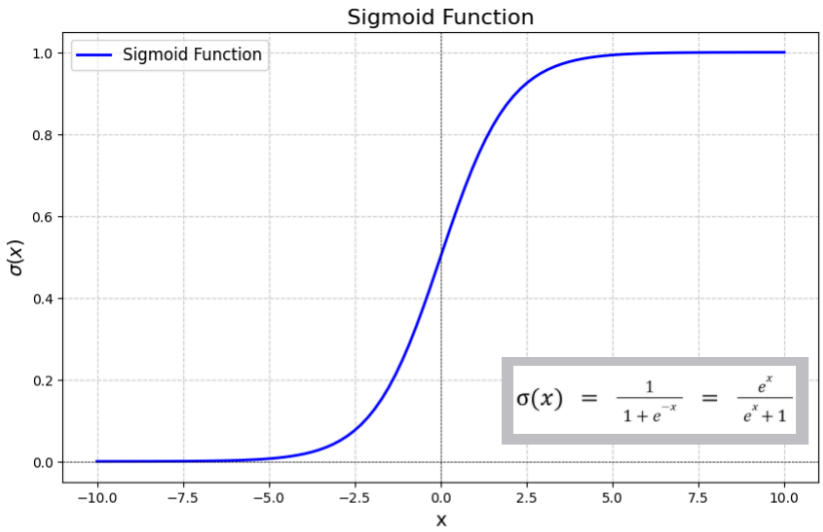
- **Classification:** where a category or discrete label is predicted (e.g., whether this email is spam or not? or which digit (0-9) is in an image).
- **Regression:** where a continuous value is predicted (e.g., predicting house prices, or a person's income).

In this lab, we will explore both types of problems. We'll start with linear regression, where the goal is to predict a continuous value, before moving to classification, where the goal is to predict which category a new observation belongs to.

Linear regression is used to model the relationship between one or more independent variables and a continuous dependent variable. It fits a straight line (or hyperplane, with multiple features) that best predicts the target value, minimizing the difference between predicted and actual values. Unlike classification, there is no threshold or category being predicted — the output is a numeric value itself (e.g., predicting a diamond's price in dollars).

Classification problems like the one in this lab can be approached in multiple ways, one of which is called Logistic Regression.

Although the term *regression* usually refers to modeling how a continuous dependent variable changes with respect to independent variables (as in linear regression), **logistic regression is actually a classification method**. The “regression” in its name comes from the fact that it models the probability of belonging to a class, not a continuous outcome itself. It can use functions like the **sigmoid function** to map predictions to values between 0 and 1, which can then be interpreted as probabilities. A threshold is applied to decide the final class label.



Sigmoid Function

Tools:

Scikit-learn is a popular Python library for machine learning. It provides easy-to-use tools for preprocessing data, training models, and evaluating their performance. In this lab, we’ll use it to build and test both a linear & logistic regression model on the Diamond Prices & Palmer Penguins datasets.

Evaluation Metrics:

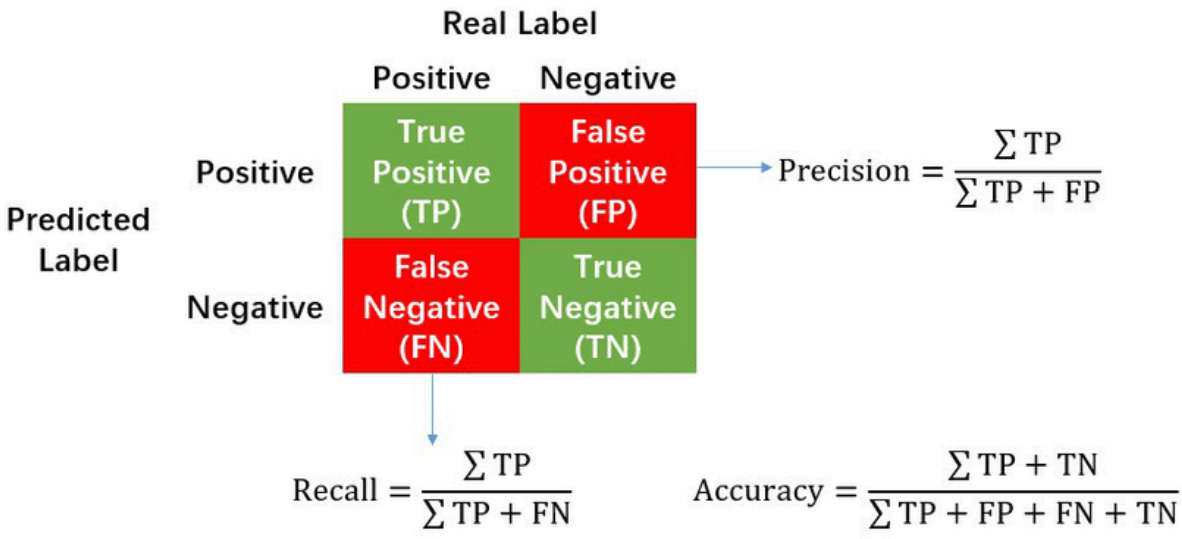
Evaluation Metrics are essential in machine learning because they tell us how well a model is performing beyond just looking at predictions. Different metrics highlight different strengths and weaknesses, helping us attain comprehensive insight about the model and avoid misleading conclusions. A few examples we will be inspecting are:

Regression Metrics:

- **MSE (Mean Squared Error)** – The average of the squared differences between actual and predicted values. Larger errors are penalized more heavily.
- **RMSE (Root Mean Squared Error)** – The square root of MSE, brought back into the original units of the target, making it more interpretable.
- **MAE (Mean Absolute Error)** – The average of the absolute differences between actual and predicted values, treating all errors equally regardless of size.
- **R^2 (R-squared)** – The proportion of variance in the target variable that the model explains, ranging from 0 to 1 (higher is better).

Classification Metrics:

- **Accuracy** – The proportion of all predictions that the model classified correctly.
- **Precision** – The proportion of positive predictions that were actually correct, in simpler terms, of everything the model predicted as positive, how many were truly positive? Or "When the model says yes, how often is it right?"
- **Recall** – The proportion of actual positives that were correctly identified by the model, in simpler terms, of all the true positives in the data, how many did the model actually find? Or "When something is actually yes, how often does the model catch it?"
- **F1 Score** – The harmonic mean of precision and recall, balancing the two metrics.



Accuracy vs Recall

Note on metrics:

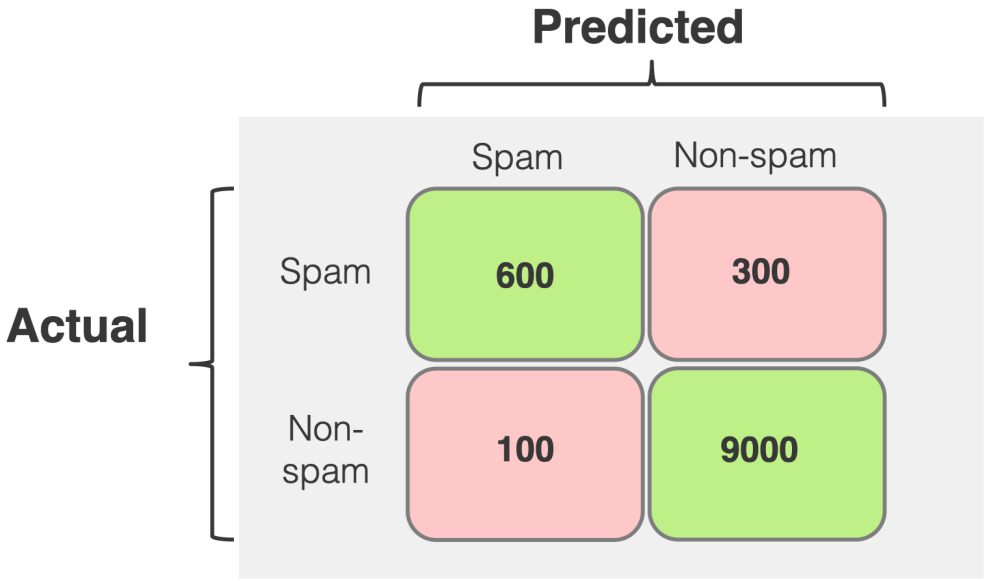
Consider Accuracy, a popular metric in ML and may seem like a great one to utilize; however, it may not be sufficient to draw conclusions. Take for example, a model that classifies cats and dogs; however, the model is so naive that it assumes "Dog" each time. Thus, when presented with a test data of 8 dogs and 2 cats, acc=80%

a test data of 3 dogs and 7 cats, acc=30% and so on, emphasizing that accuracy is not the best metric to be used on its own in some circumstances where the test dataset is not balanced, and that follows for the rest of the metrics, which have different trade-offs for their usage.

Confusion Matrix:

Precision and recall can be visualized using a **confusion matrix,** which is a table that summarizes how well a **classification model** performs by comparing its **predicted labels** to the **true labels**.

It shows where the model is correct and where it makes different kinds of mistakes.



Confusion matrix of email classification

By the end of this lab, you will learn to:

1. Load a dataset and split it into training and testing sets
2. Train regression and classification models using Scikit-learn
3. Evaluate regression models using performance metrics like:
    - MSE, RMSE, MAE (error magnitude)
    - R^2 (overall fit/variance explained)
4. Evaluate classification models using performance metrics like:
    - Accuracy (overall correctness)
    - Precision (how many predicted positives were correct)
    - Recall (how many actual positives were found)
    - F1 Score (balance between precision and recall)

We will use the **Diamond Prices** dataset for the Guided Task, and the Palmer Penguins dataset for the graded task.

Required Readings:

1. Scikit-Learn User Guide – Supervised Learning
2. Train-Test Split Documentation
3. Metrics and Scoring
4. Linear Regression
5. Logistic Regression
6. Overview of Accuracy, Precision and Recall

Required Watch List:

1. Supervised and unsupervised Learning overview
2. Logistic Regression

Part 2: In-Lab Task: Notebook Link

1. Set Up Your Environment.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score, accuracy_score,
    precision_score, recall_score, f1_score, confusion_matrix,
    classification_report
)
```

2. Add the Diamonds Prices Dataset to your Kaggle notebook and do some basic cleaning steps. We will also create a binary column called ideal_cut for our logistic regression task.

```
df =
pd.read_csv('/kaggle/input/datasets/nancyalaswad90/diamonds-prices/Diamonds
Prices2022.csv', index_col=0)

df = df.drop_duplicates()

cut_order = ['Fair', 'Good', 'Very Good', 'Premium', 'Ideal']
color_order = ['J', 'I', 'H', 'G', 'F', 'E', 'D']
clarity_order = ['I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2', 'VVS1', 'IF']

df['ideal_cut'] = (df['cut'] == 'Ideal').astype(int)

encoder = OrdinalEncoder(categories=[color_order, clarity_order, cut_order])
df[['color', 'clarity', 'cut']] = encoder.fit_transform(df[['color',
'clarity', 'cut']])
```

3. Define features and labels for the Classification Task:

```
#We will drop cut so we don't have any data leakage and predict whether the
cut is ideal or not

X = df[['carat', 'color', 'clarity', 'depth', 'table','price','x','y','z']]
```

```
y = df['ideal_cut']
```

4. Split the dataset into training and testing.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

5. Train a classification model.

```
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)
```

6. Evaluate the model.

```
y_pred = logistic_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

7. Visualize the confusion matrix.

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

8. Start the regression task by selecting the correct feature set and target variable,

then split the dataset.

```
#We will drop ideal_cut and use the normal cut column in order to predict
the price
X = df[['carat', 'color', 'clarity', 'depth', 'table','cut','x','y','z']]
y = df['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

9. Train the linear regression model, and predict.

```
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred = linear_model.predict(X_test)
```

10. Evaluate using mse, rmse, mae, r^2:

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R²: {r2:.4f}")
```

Part 3: Graded Task

Task Description:

1. Use the **Palmer Penguins Dataset** to train a classification model that predicts

whether the penguin is a male or female.

2. Report the following metrics for the classification model:
    - Accuracy
    - Precision
    - Recall
    - F1 Score
3. Display a **confusion matrix** using `seaborn`.
4. Use the dataset to train a regression model that predicts the body_mass_g feature for the penguin.
5. Report the following metrics for the regression model:
    - RMSE
    - MSE
    - MAE
    - R^2
6. Choose one of the above models and attempt to improve its evaluation metrics, and compare the results with a visualization technique.

Deliverables:

Submit a notebook with the completed above task that **must** include:

- The metrics of **each** model
    - Accuracy
    - Precision
    - Recall
    - F1 Score
    - RMSE
    - MSE
    - MAE
    - R^2
- A comparison visualization between the baseline and improved models of your choice.

Submit an accessible notebook link containing all the completed tasks listed above via the form. The submission **deadline** is by the end of your lab.